BUILT · DEPLOYED · VERIFIED

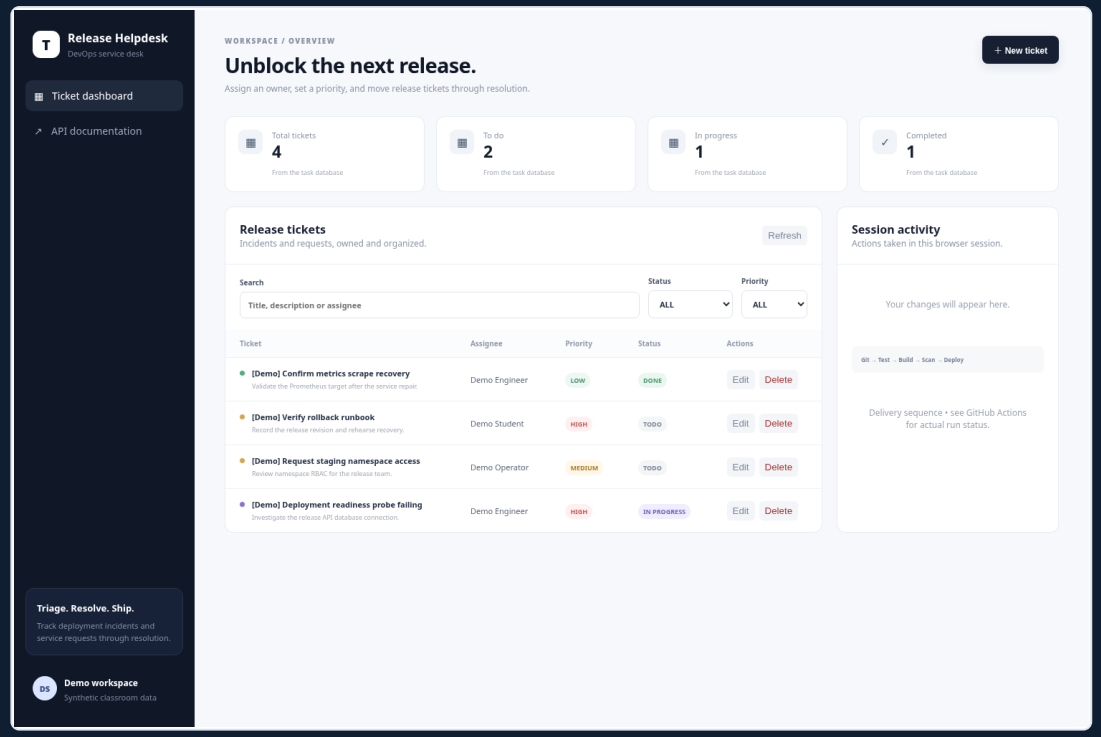
Release Helpdesk

From Git commit to monitored Kubernetes.

End-to-end release engineering

React · FastAPI · PostgreSQL · October 2026

WATCH THE RECORDED DEMO



ACTUAL AWS APPLICATION · CAPTURED 16 OCT 2026

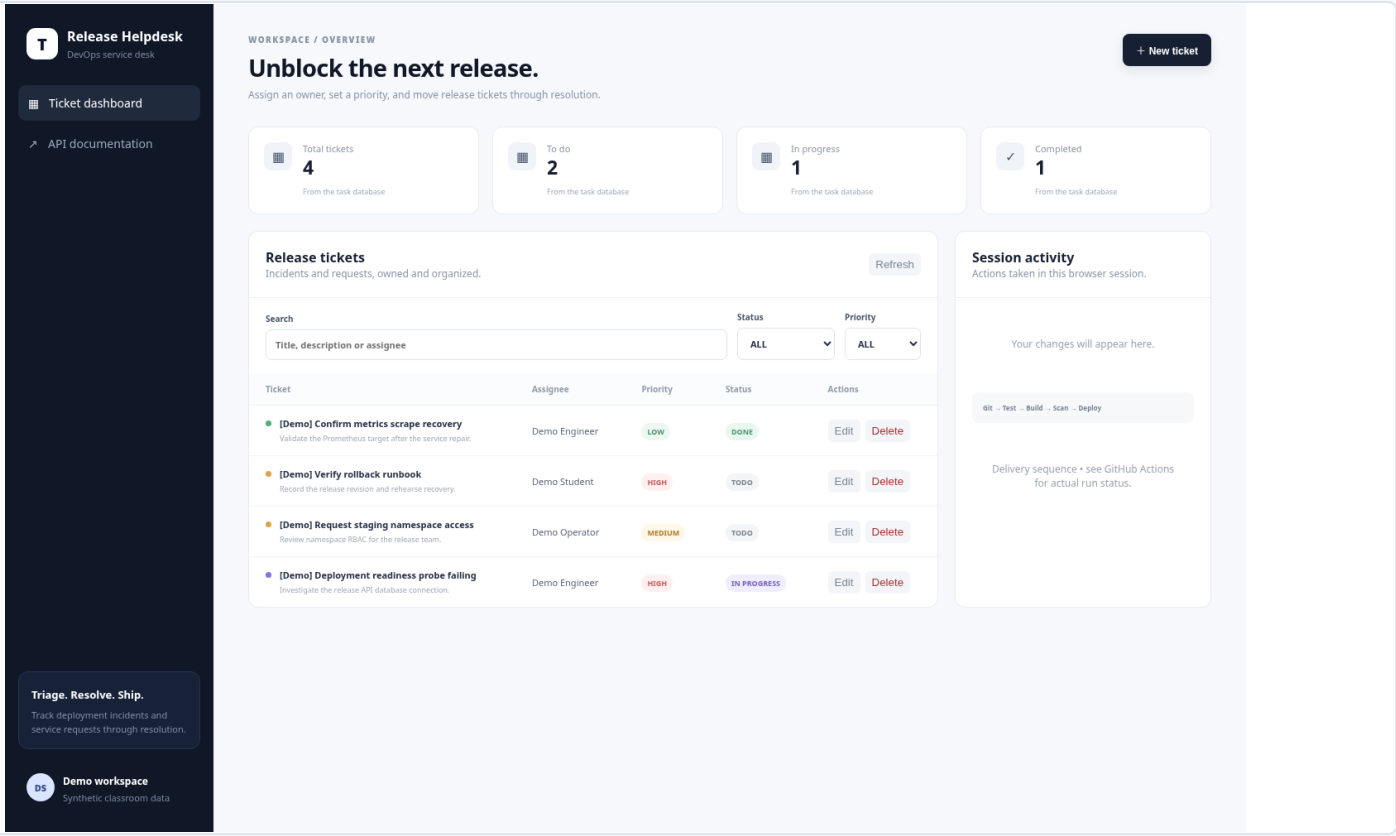
19
API test cases

2 → 4 → 2
Ready replicas

66
IaC resources

A focused desk for release operations.

Capture the incident, assign an owner, and move the next release forward.



Actual application through AWS ALB · full original screenshot

Evidence · Real AWS browser walkthrough and responsive UI

01 Capture

Create a release ticket with context, priority and an assignee.

02 Triage

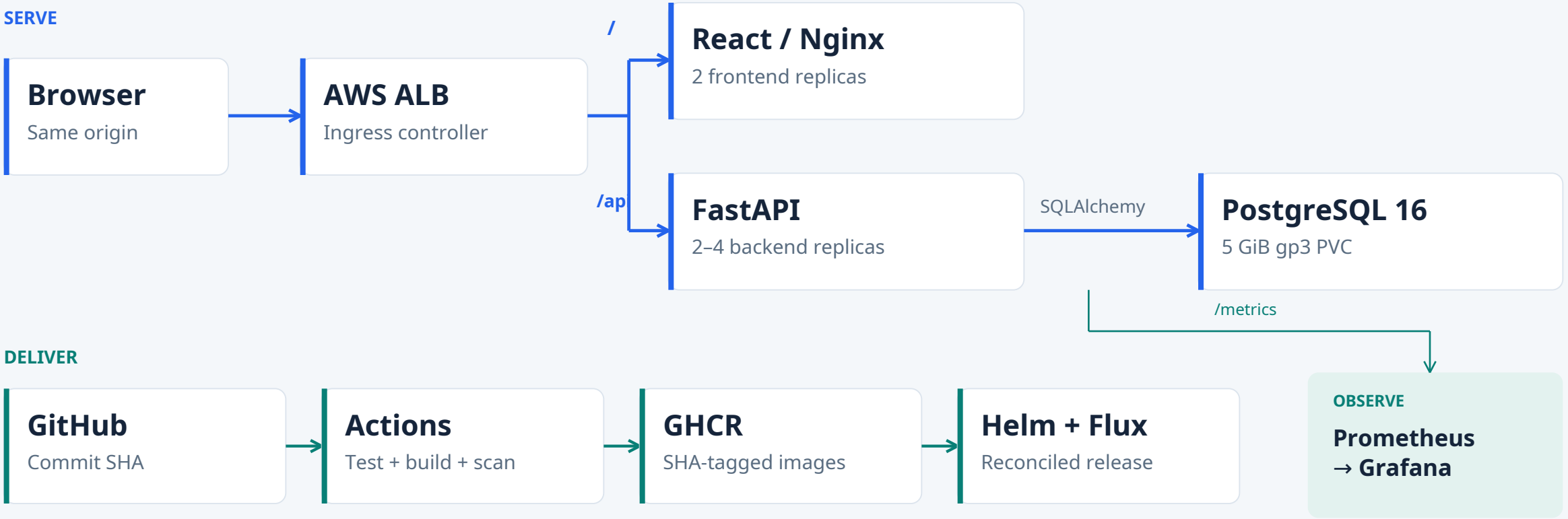
Search and filter the queue; edit ownership and status.

03 Resolve

Complete or delete tickets; see database-backed counts update.

One system. Three connected paths.

Request serving, release delivery and observability share the same deployed application.



Source · Implemented architecture and routing

Correctness starts before deployment.

A tested API contract and one migration authority protect the data layer.

19

passing API test cases

Isolated test database

SIX REST OPERATIONS

```
GET /api/tasks
GET /api/tasks/{id}
GET /api/tasks/stats
POST /api/tasks
PUT /api/tasks/{id}
DELETE /api/tasks/{id}
```

DB

Alembic only

Schema revision 0001_create_tasks; migrations run before traffic is ready.

IO

Data survives

4 records survived a PostgreSQL Pod restart.

`/health`

Process is alive

`/ready`

Database + tasks table are usable

DB outage: 200 → 503 → 200

Evidence · PostgreSQL tests, migration and restart persistence

The complete stack starts with one command.

Compose encodes dependencies; runtime containers execute as non-root users.

```
docker compose up --build -d --wait
```

PostgreSQL
Healthy

→

Alembic
Completed

→

FastAPI
Ready

→

React / Nginx
Serving

Backend runtime

Python 3.13 → Uvicorn

USER 10001

Frontend multi-stage build

Node 24 build → unprivileged Nginx

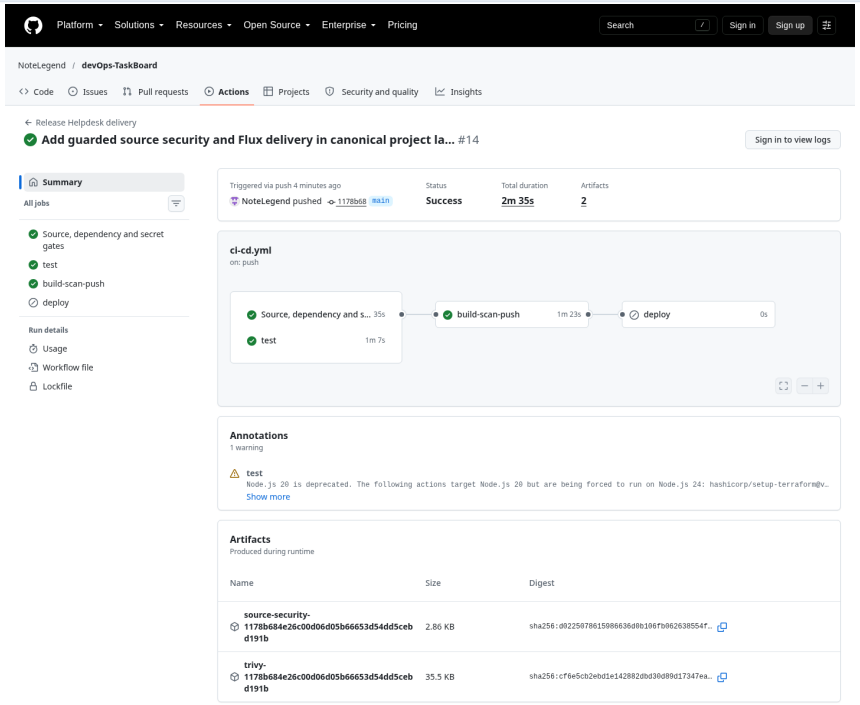
USER 101

Only compiled assets enter the frontend runtime.

Evidence · Real Compose stack, images and runtime users

A guarded path from commit to cluster.

Tests and source-security gates must succeed before either image is published.



Actual hosted run · test, source security and publication passed

01 Validate

PostgreSQL tests, frontend build and source/dependency/secret gates

02 Build, scan & publish

Both images → Trivy → GHCR with immutable full-SHA tags

03 Promote through Git

Pin the verified SHA → Flux → Helm → drift correction

Meaningful, traceable Git history

Evidence · Actual integrated source-security and image publication run

PRs validate without publishing; AWS deploy remains disabled after teardown.

Security findings lead to repairs, then a release.

Source analysis, dependency audits and secret checks run before image publication.

BACKEND IMAGE

0 **HIGH / CRITICAL findings reported**

FRONTEND IMAGE

0 **HIGH / CRITICAL findings reported**

Certifi trust-store advisory → upgrade to 2026.10.19 → rebuild and rescan

01 **Source + dependencies**

Bandit / Semgrep; pip-audit / npm audit.

02 **Secrets + final images**

Full-history Gitleaks; both Trivy severity gates.

Evidence · Fresh source and image reports for release 1178b68

AWS infrastructure is declared in Terraform.

Mumbai / ap-south-1 · EKS 1.35 · two managed m7i-flex.large workers



66

Terraform resources created

01 **Pinned configuration**

Modules, providers, add-ons and node AMI are versioned.

02 **Explicit account guard**

Approved Free-plan credit use; no Paid-plan upgrade.

Evidence · Actual approved plan, apply and AWS Console captures

Helm turns the stack into a repeatable release.

One versioned chart connects workloads, storage, probes, Services and Ingress.



Real AWS resource listing · taskboard namespace

`Ingress: / → frontend:8080 /api → backend:8000`

Evidence · Actual AWS Pods, Services, PVC, Helm release and Ingress

01 Replicated workloads

2 frontend Pods; backend HPA starts at 2 and can reach 4.

02 Persistent database

PostgreSQL StatefulSet + encrypted 5 GiB gp3 volume.

03 Controlled release

One migration Job with advisory lock; readiness gates traffic.

Committed intent keeps the running service honest.

Flux reconciles this application and Helm chart from the public repository on local Kubernetes.



Actual controller state and recorded verification · local kind

2 → 1 → 2

Manual frontend drift → automatic repair

01 Promote from Git

Scanned image SHA + gitops-v2 header arrived through Flux.

02 Automatic recovery

2 frontend replicas restored in 34 seconds.

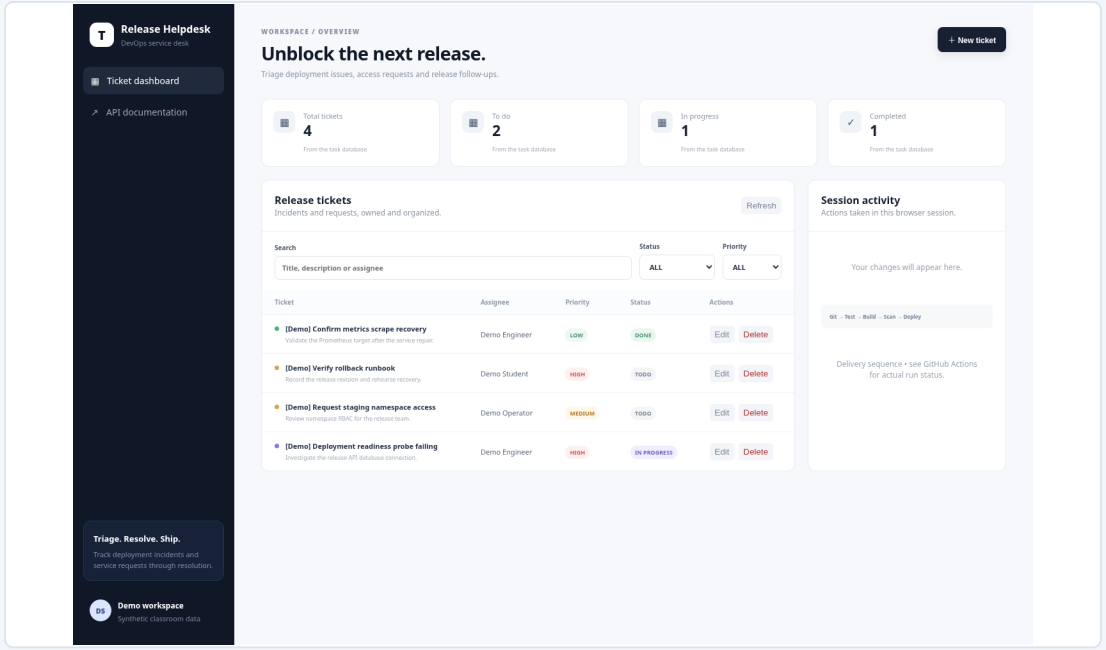
Registry digests match both running application images.

Evidence · Actual Flux promotion, drift recovery and digest checks

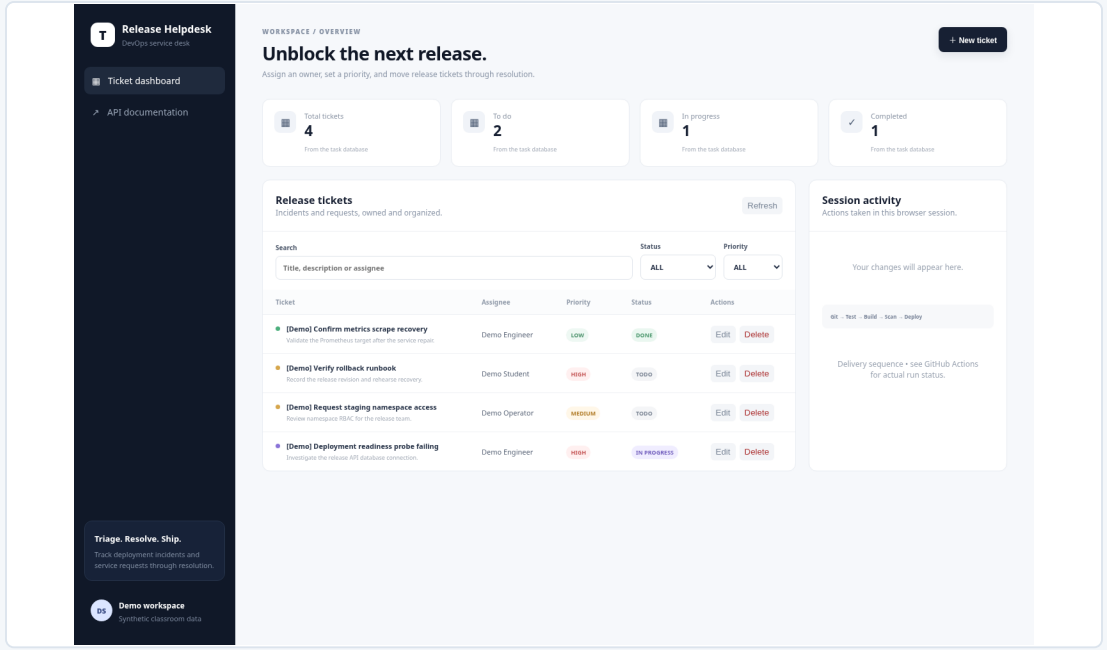
One small change proves the delivery chain.

A visible triage-help change reached the real AWS application through CI.

BEFORE · 3ffe809



AFTER · 179ad20

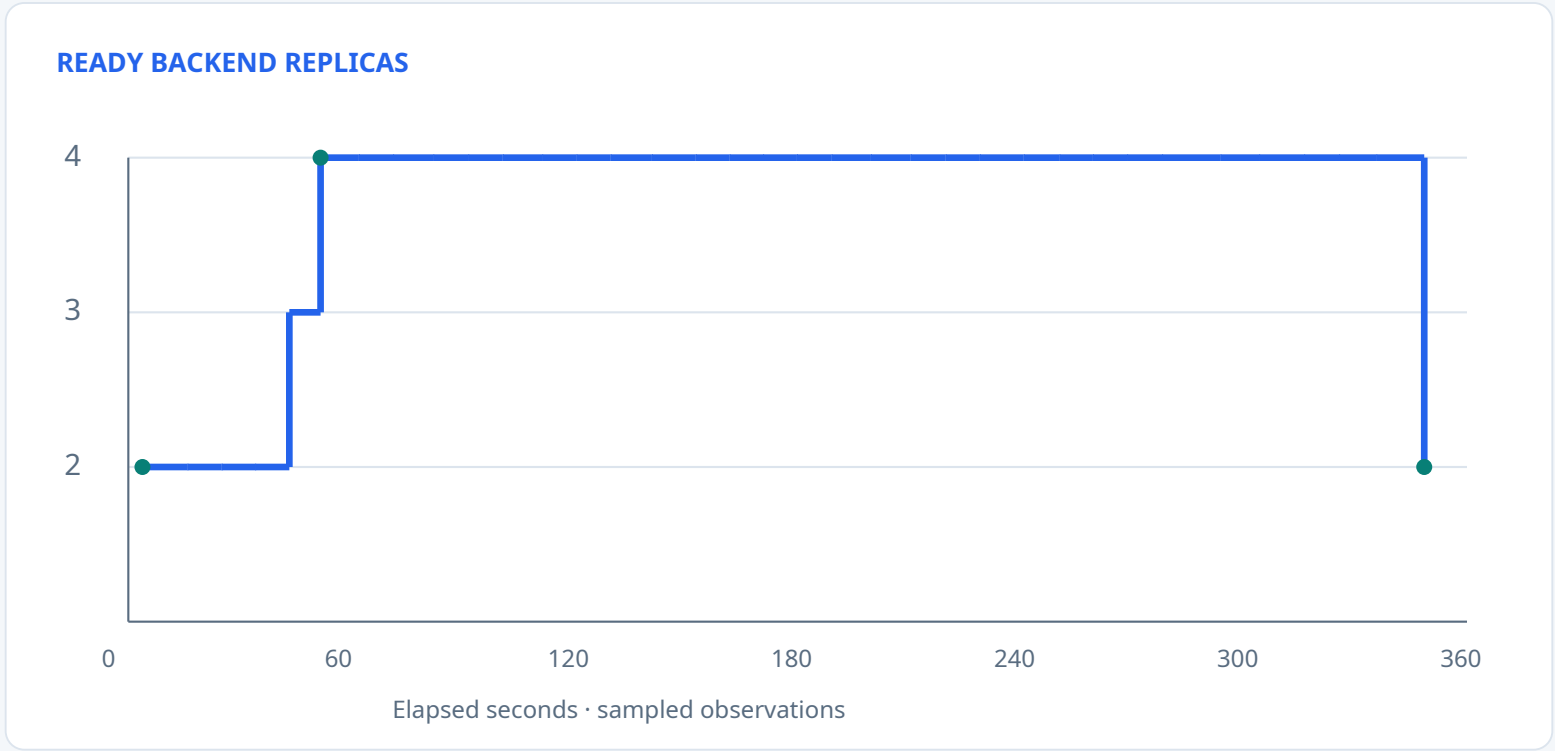


Commit → tests → build → scan → SHA-tagged GHCR images → OIDC / Helm → visible update

Evidence · Successful AWS update for commit 179ad20 · click to open

CPU demand produced real scaling and recovery.

Observed on AWS: ready backend replicas increased under load, then recovered automatically.



2 → 4 → 2

Ready replicas, measured

68,466

successful requests

0

failed load requests

180.1 s

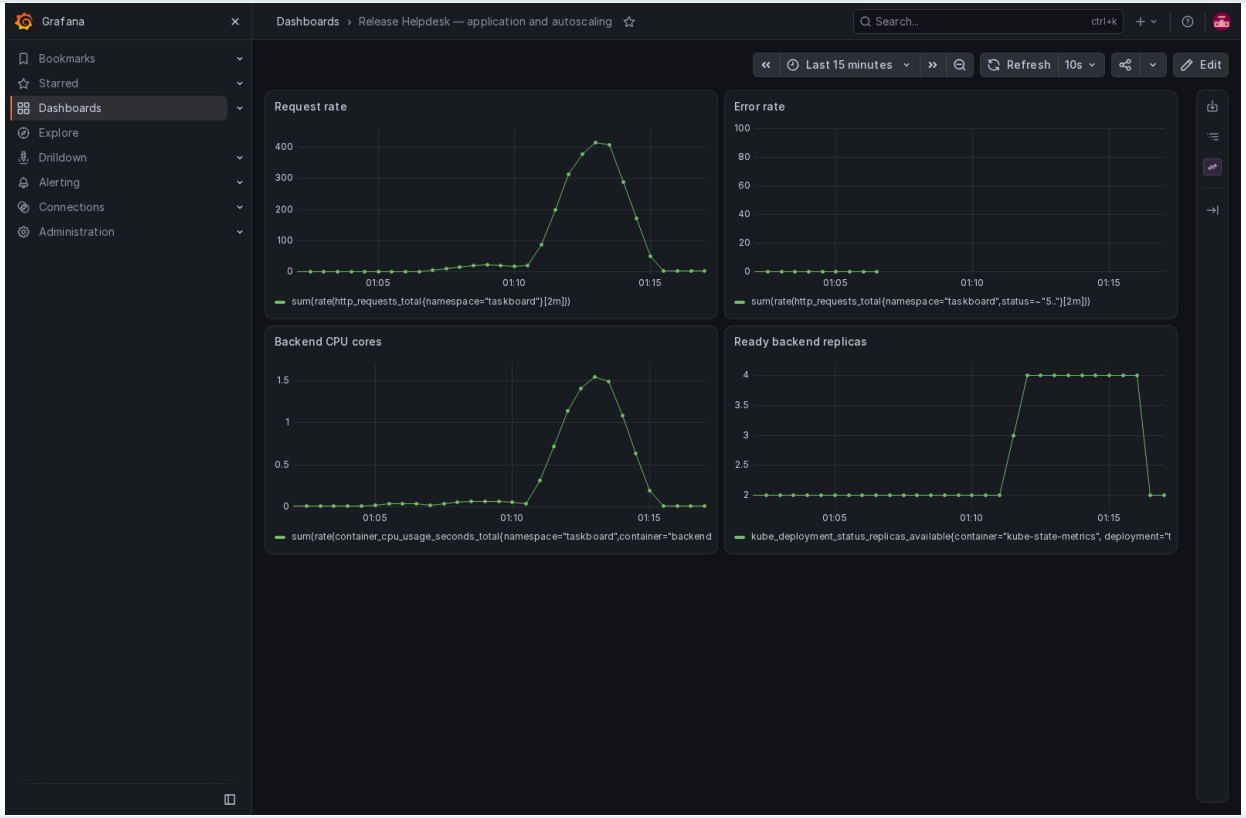
24-worker in-cluster load

HPA: 60% CPU target · 100m request / Pod · min 2, max 4 · peak 442% of requested CPU

Evidence · Chart drawn from actual timestamped HPA observations

See traffic, latency and errors in one place.

Application instrumentation → ServiceMonitor → Prometheus → authenticated Grafana.



Actual recorded AWS dashboard · HTTP traffic, errors, latency and replicas

Evidence · Authenticated AWS Grafana dashboard with populated panels

01 Instrument

Request counter + latency histogram; route-template labels.

02 Scrape

Backend /metrics; real Prometheus application targets UP.

03 Inspect privately

Grafana via private port-forward with generated credentials.

Two failures. Two traceable recoveries.

Controlled exercises ran in an isolated namespace; diagnosis followed observable evidence.

IMAGE FAILURE

ImagePullBackOff

01 Pod not Ready

02 Describe Pod + events

03 Nonexistent image tag

Correct tag → rollout → Ready

SERVICE FAILURE

No reachable backend

01 Service has no endpoints

02 Compare selector + Pod labels

03 Selector does not match

Correct selector → HTTP success

Evidence · Actual AWS failure symptoms, diagnosis and recovered HTTP responses

The deployment was verified, then removed.

Cloud evidence was pushed and checked before the authorized teardown.



Actual Terraform destroy completion · 66 resources destroyed

Recorded cleanup: 16 Oct 2026 · KMS key disabled / pending deletion · billing settlement may lag

Evidence · Destroy completion and recorded residual-resource checks

66

Terraform resources destroyed

01 Residual checks passed

Project EKS, VPC, ALB, workers, NAT and retained disks absent.

02 Deployment disabled

DEPLOY_ENABLED=false; AWS application URL is historical.

Tested. Reconciled. Observable.

01

Reproducible

Compose locally. Terraform for AWS. One chart for the application.

02

Guarded

Source and dependency checks, secret scans and immutable image gates.

03

Self-correcting

GitOps restores intent. HPA reacts to CPU. Metrics explain behavior.

Next production steps

End-user identity · HTTPS + DNS · managed database backups · service alerts

github.com/NoteLegend/devOps-TaskBoard

Review · Executed results and reproducible project source